SET 19

PART A — Short questions and answers

Q1. Explain a one-to-one relationship using CAR and REGISTRATION_CERTIFICATE.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Describe how the foreign key CarID in RENTAL ensures that only existing cars can be rented.

Answer: A foreign key maintains a valid relationship by referring to a key in another table.

Q3. Describe a full functional dependency using (Customer_ID, Car_ID) → Total_Cost.

Answer: Book_ID → Title means each Book_ID determines exactly one book title.

Q4. Describe the purpose of a self join using the CAR table as an example.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Describe the effect of grouping by two columns, Brand and Status.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Describe when the TOO_MANY_ROWS exception is raised.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Explain what schema-less means in MongoDB.

Answer: MongoDB stores records as BSON documents inside collections.

Q8. Explain the use of the $push operator to add an element to an array field.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Interpret the MongoDB query: db.cars.find({Status: "Available"}).sort({DailyRate: -1})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Explain the purpose of DISTINCT in the RETURN clause.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A car rental agency maintains a Car Rental System to store customer details, car details, and rental

1(a) ER design:

• **CUSTOMER**(CustomerID, CustomerName, LicenseNo, Phone); primary key: **CustomerID**.

• **CAR**(CarID, Brand, Model, DailyRate, Status); primary key: **CarID**.

• **RENTAL**(RentalID, CustomerID, CarID, PickupDate, ReturnDate, TotalCost); primary key: **RentalID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE CUSTOMER (
  CustomerID NUMBER PRIMARY KEY,
  CustomerName VARCHAR2(100),
  LicenseNo VARCHAR2(100),
  Phone VARCHAR2(100)
);
CREATE TABLE CAR (
  CarID NUMBER PRIMARY KEY,
  Brand VARCHAR2(100),
```

```
  Model VARCHAR2(100),
  DailyRate NUMBER,
  Status VARCHAR2(100)
);
CREATE TABLE RENTAL (
  RentalID NUMBER PRIMARY KEY,
  CustomerID NUMBER,
  CarID NUMBER,
  PickupDate DATE,
  ReturnDate DATE,
  TotalCost NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM RENTAL x
JOIN CUSTOMER a ON x.CustomerID = a.CustomerID
JOIN CAR b ON x.CarID = b.CarID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```